3. CODE

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process\n");
        printf("PID  : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        sleep(10);
    }
    else
    {
        printf("Parent Process\n");
        printf("PID  : %d\n", getpid());

        wait(NULL);
    }

    return 0;
}
```

<u>OUTPUT:-</u>

```
praneeth@PRANEETH:/mnt/c/Users/praneeth$ gcc 3.c -o 3
praneeth@PRANEETH:/mnt/c/Users/praneeth$ ./3
Parent Process
PID  : 607
Child Process
PID  : 608
PPID : 607
```

4. CODE

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;

    for(i=0;i<3;i++)
    {
        if(fork()==0)
        {
            printf("Child %d PID=%d\n",i+1,getpid());
            return 0;
        }
    }

    for(i=0;i<3;i++)
    {
        wait(NULL);
    }

    printf("All children completed.\n");

    return 0;
}
```

OUTPUT:-

```
praneeth@PRANEETH:/mnt/c/Users/praneeth$ nano 4
praneeth@PRANEETH:/mnt/c/Users/praneeth$ gcc 4.c -o 4
praneeth@PRANEETH:/mnt/c/Users/praneeth$ ./4
Child 1 PID=691
Child 2 PID=692
Child 3 PID=693
All children completed.
praneeth@PRANEETH:/mnt/c/Users/praneeth$
```